Algebraic Study of the 3-Tensor Isomorphism Problem

David Pulido Cornejo

School of Computer and Communication Sciences

Semester Project

June 2025

Responsible
Prof. Serge Vaudenay
EPFL / LASEC

Supervisor
Laurane Marco
EPFL / LASEC

Contents

1. Introduction

As quantum computing advances, it threatens current public-key cryptographic systems. Classic cryptographic primitives, such as RSA, could be easily broken with a sufficiently powerful quantum computer. For instance, RSA, whose hardness relies on integer factoring, could be overcome by Shor's [16] or more recently, Regev's [15] algorithm in polynomial time. This threat has driven the research, development and standardization of post-quantum cryptography: new cryptographic algorithms believed to be secure against both quantum and classical attacks.

Notably, the U.S. National Institute of Standards and Technology (NIST) initiated an open competition to evaluate and standardize quantum-resistant algorithms. After several evaluation rounds, NIST has identified a handful of leading candidates for standardization, for example, the lattice-based CRYSTALS-Kyber (for key encapsulation [4]) and CRYSTALS-Dilithium (for digital signatures [11]), as well as the hash-based SPHINCS+ signature scheme [1].

On the other hand, recognizing the importance of diversity in cryptographic assumptions, NIST has reopened the call for alternate digital signature schemes that are not lattice-based and that offer practical benefits like shorter signatures and faster verification [14]. Among these emerging alternatives are graph-based cryptographic constructions, which draw on the hardness of various isomorphism problems. In particular, two recent digital signature candidates, MEDS [7] and ALTEQ [3], base their security on problems equivalent to the 3-Tensor Isomorphism Problem (3-TIP), which asks, given two 3-tensors known to be isometric, to compute the linear maps (an isometry) that transform one tensor into the other.

Almost immediately after the submission of these signature schemes, Ran and Samardjiska [14] took the cryptanalysis of the 3-TIP in a new direction. Following a common approach in graph-based methods, their algorithm identifies invariants in the graphs derived from 3-tensors to recover the secret isometry. However, their approach differs by modeling this invariant as a system of non-linear equations and solving it accordingly. In their study, they prove that a projective triangle (three mutually adjacent vertices in the associated tripartite graph) appears with probability about $1/q$ (q being the field size) even in random, supposedly hard instances. Furthemore, they devised a compact system of quadratic equations using these algebraically computed projective triangles that when solved, is able to reconstruct the full hidden isometry $(A, B, C) \in GL(n, q) \times GL(m, q) \times GL(k, q)$ between two isomorphic tensors.

Their attack, which recovers ALTEQ level-I keys in under half an hour on commodity hardware [14], highlighted how tiny, rare-looking sub-structures can undermine seemingly hard instances. This observation motivates the present study: we ask how many such short cycles exist, how their abundance shifts when the tensor dimensions diverge, if a generalization for higher-length closed walks is possible, and how software can help researchers explore these and other patterns at scale.

1.1. Related work

Research on the 3-Tensor Isomorphism Problem (3-TIP) and its cryptographic instantiations has accelerated since the appearance of MEDS and ALTEQ. Below we review the lines of work most pertinent to this report.

The birthday-paradox methodology introduced by Bouillaguet et al. for the Isomorphism of Polynomials [5] was transplanted to 3-TIP by Narayanan, Qiao and Tang [12]. Their paper devises new, easily computed isomorphism invariants for Matrix Code Equivalence (MCE) and Alternating Trilinear Form Equivalence (ATFE). Concretely, they (i) collect points whose associated bilinear forms have a prescribed rank, (ii) hash those points with a distinguishing function f that collides only on isometric pairs, and (iii) use the collision (a, b) to reconstruct the isometry. Because only $O(\sqrt{N})$ points need be stored, the algorithm runs in $O(q^{n/2})$ operations for the case where $n = m = k$ and enjoys a cubic quantum speed-up. The price, however, is large memory and heavy randomization: both time and space grow with the square root of the instance size, and the method cannot proceed without maintaining the collision lists in RAM.

In contrast, the triangle-based attack of Ran and Samardjiska [14] takes an entirely different route. They showed that a random tensor graph contains a projective triangle with probability $1/q$. Once such a triangle is found, all three linear maps are recovered by solving a nonlinear system of equations. Unlike the birthday-paradox-based techniques, the whole attack is therefore algebraic and deterministic. In practice, they recover an ALTEQ level-I secret key in roughly half an hour, cutting about 60 security bits from the original estimate.

Concerning non-graph-based approaches, Couvreur and Levrat recently introduced the "Highway to Hull" algorithm [8], which reduces general MCE to the conjugacy case $P = Q$ and then applies a hull invariant followed by a list-collision search. Their technique matches Narayanan et al.'s $O(q^{k/2})$ complexity when $k = n = m$ but, crucially, extends to highly rectangular codes, broadening the parameter range affected by practical attacks.

Beyond explicit cryptanalysis, a body of work, most notably Grochow and Qiao's TI-completeness results, situates 3-TIP, MCE and ATFE in a single equivalence class together with cubic-form, p-group, among other isomorphism problems [9]. These reductions motivate the systematic study of tensor graphs undertaken here, because progress (or weakness) in one representative propagates to many others.

1.2. Aims

Taken together, these studies demonstrate both the promise and uncertainty of graph-based constructions. The objective of this project is then to generalize and develop new graph-based techniques to solve the 3-TI problem inspired by the hybrid projective triangle approach.

1.3. Our contribution

This report makes two main complementary contributions that aim to advance the algebraic study of the 3-TIP. First, we characterize the shape of tensor

graphs. In section 4, we generalize the study of triangles in $\mathcal{G}(\mathcal{C})$ and extend the analysis to all closed walks of length $\ell \geq 3$, proving the ℓ-independent and dimension independent bound $\mathbb{E}[M_\ell(C)] = \Theta(1/q)$. Then, through a counting argument we reveal that $\Theta(2^\ell/\ell)$ algebraic systems of quadratic equations suffice to describe every walk type of length ℓ, making triangles the clear sweet-spot for practical isometry attacks. Empirically we observe a qualitative geometric transition when at least one of the three dimensions differs (e.g. $n = m \neq k$): the giant core of a tensor graph fractures into many tree-like micro-components, a phenomenon documented with high-resolution visualizations.

Complementing these theoretical insights, we release an open-source tensor-graph toolkit that turns theory into practice. We implement a modular program that (i) generates random tensors, (ii) constructs a tensor graph, (iii) filters vertices by degree, (iv) highlights cycles, (v) applies arbitrary isometries, and (vi) exports publication-quality PNGs. A single command thus "spins up" a full experiment pipeline. The naive edge-enumeration strategy still costs $O(q^{2n}n^3)$ operations, so the tool is intentionally made for the small scale: perfect for $n \leq 5, q \leq 17$ toy instances but unable to render cryptographic-size graphs where $n = 30, q = 2039$. All source code and reproducibility scripts are publicly available at `https://github.com/david-pulido/3TI_Graph_Visualizer` and the respective design choices are discussed in section 6.

Together, these theoretical, algorithmic and software components provide a novel end-to-end framework for measuring, drawing and attacking tensor graphs, potentially laying empirical and conceptual groundwork for future work on post-quantum schemes based on the 3-TIP.

2. Preliminaries

Before delving into size, shape, algorithms, and implementations, we will fix language and notation. We will begin by formalizing the notation for 3-tensors over a finite field and the projective points we will later view as graph vertices. We will then state the 3-Tensor Isomorphism Problem (3-TIP) and recall its equivalence to several other isomorphism and code-equivalence problems. Next, we will refresh core graph-theoretic notions (walks, cycles, complete and multi-partite graphs), which we will translate into algebraic constraints later on. With those pieces in place, we will show how every tensor spawns a tripartite graph and justify the relevance of this approach in the context of the 3-TIP. Finally, we will anchor the discussion in practice by listing the concrete parameter sets used by the MEDS and ALTEQ signature schemes, whose security levels motivate the asymptotic regimes analyzed in subsequent sections.

2.1. Tensors

Let U, V, W be finite-dimensional vector spaces over $\mathbb{F}_q$ of dimensions n, m, k respectively. In the context of this report a 3-tensor $\mathcal{C} \in (U \otimes V \otimes W)^*$ will be considered as a trilinear form represented as a triple indexed array where $\mathcal{C}_{i,j,l} \in \mathbb{F}_q$ denotes the (i, j, l)-th entry of the tensor with $1 \leq i \leq n, 1 \leq j \leq$

$m, 1 \leq l \leq k$. For practical reasons, whenever explicitely written down, a tensor $\mathcal{C}$ will be layed out as an n-dimensional vector of matrices $\mathcal{C}_i \in \mathcal{M}_{m\times k}(\mathbb{F}_q)$ for $0 \leq i \leq n$.

Example 2.1. Let

$$\mathcal{T} := \left[\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \quad , \quad \begin{pmatrix} 5 & 6 \\ 7 & 8 \end{pmatrix} \right] \in (\mathbb{F}_9^2 \otimes \mathbb{F}_9^2 \otimes \mathbb{F}_9^2)^*$$

The value 7 is located at position $(2, 2, 1)$ noted $(\mathcal{T}_2)_{2,1} = \mathcal{T}_{2,2,1}$

Similarly, for $(u, v, w) \in (U \times V \times W)$, u_i, v_i, and $w_i \in \mathbb{F}_q$ will represent the i-th entry of u, v, and w respectively. Given this notation, we can explicitely evaluate a tensor:

$$\forall (u, v, w) \in (U \times V \times W), \mathcal{C}(u, v, w) := \sum_{i=1}^{n} \sum_{j=1}^{m} \sum_{l=1}^{k} \mathcal{C}_{i,j,l} \cdot u_i \cdot v_j \cdot w_l$$

Furthermore, we will denote:

- $\{e_i^U\}_{i=1}^n$, $\{e_j^V\}_{j=1}^m$, and $\{e_l^W\}_{l=1}^k$ the set of vectors in the canonical basis of U, V and W respectively
- For $(u, v) \in U \times V, \mathcal{C}(u, v, -) = 0$ means that $\forall w \in W, C(u, v, w) = 0$. Similarly for $\mathcal{C}(-, v, w)$ and $\mathcal{C}(u, -, w)$
- In the coming paragraphs, $\mathbb{P}(V)$ will denote the projective space associated with V.
- $\hat{v} \in \mathbb{P}(V)$ an element in the projective space and $v \in V$ a representative of $\hat{v}$

Now that the object of interest has been defined and the notation has been layed out, we can proceed to describe the problem in question.

2.2. The 3-Tensor Isomorphism Problem

Definition 2.1 (3-TI Problem)**.** Let $\mathcal{C}, \mathcal{D} \in (U \otimes V \otimes W)^*$. The 3-Tensor Isomorphism (3-TI) Problem asks to find an isometry $(A, B, C) \in \mathrm{GL}(U) \times \mathrm{GL}(V) \times \mathrm{GL}(W)$ such that,

$$\forall (u, v, w) \in U \times V \times W, \mathcal{C}(Au, Bv, Cw) = \mathcal{D}(u, v, w)$$

In algebraic terms, given two 3-tensors we are asked to determine whether they belong to the same orbit by the right group action of $\mathrm{GL}(U) \times \mathrm{GL}(V) \times \mathrm{GL}(W)$ over $(U \otimes V \otimes W)^*$.

Example 2.2. Let $\mathcal{T}$ and $\mathcal{T}'$ be two $(2,2,2)$-dimensional 3-tensors over $\mathbb{F}_5$ defined as follows:

$$\mathcal{T} := \left[\begin{pmatrix} 0 & 0 \\ 1 & 4 \end{pmatrix} \quad , \quad \begin{pmatrix} 4 & 3 \\ 0 & 2 \end{pmatrix} \right]$$
$$\mathcal{T}' := \left[\begin{pmatrix} 3 & 0 \\ 2 & 2 \end{pmatrix} \quad , \quad \begin{pmatrix} 3 & 3 \\ 4 & 4 \end{pmatrix} \right]$$

$\mathcal{T}$ and $\mathcal{T}'$ are isomorphic. Indeed, $\mathcal{T} \circ (A, B, C) \equiv \mathcal{T}'$, for

$$A := \begin{pmatrix} 3 & 3 \\ 3 & 0 \end{pmatrix} \qquad B := \begin{pmatrix} 3 & 2 \\ 4 & 3 \end{pmatrix} \qquad C := \begin{pmatrix} 1 & 2 \\ 0 & 1 \end{pmatrix}$$

We can verify this by computing $\sum_{i,j,l} \mathcal{T}_{i,j,l} \cdot A_{i,p} \cdot B_{j,q} \cdot C_{l,r}$ for $1 \leq p, q, r \leq 2$ and comparing against ${\mathcal{T}'}_{p,q,r}$.

As Grochow and Qiao suggested, the 3-TIP defines in on itself a complexity class [9]. In particular, they notice that the d-Tensor Isomorphism Problem is polynomial time equivalent to the 3TIP. Similarly, the Alternating Trilinear Form Equivalence Problem (ATFE) and Matrix Code Equivalence Problem (MCE) (from which the signature schemes AFTE [3] and MEDS [7] are based on) are equivalent to the 3-TIP.

2.3. Graph Theory

Since our study of the 3-TIP is both algebraic and graph-based, we will give a quick primer on graph theory. These definitions and results can also be found in [17].

Definition 2.2. An unweighted graph $\mathcal{G}$ is defined as a pair $(\mathcal{V}, \mathcal{E})$ where

- $\mathcal{V}$ is a set of nodes or, interchangeably, vertices.
- $\mathcal{E}$ is a set of edges connecting pairs of vertices. In particular, it is a subset of $\mathcal{V}^2$. We say that $\mathcal{G}$ is undirected if each edge links two vertices symmetrically.

Furthermore, the order of a graph corresponds to the size of its vertex set.

We focus exclusively on finite graphs, meaning those with a finite number of vertices. Moreover, for any graph of order n, we adopt the canonical labeling of its vertices as $1, 2, \ldots, n$. Note that any other labeling can be transformed into this canonical form through an appropriate relabeling isomorphism.

Definition 2.3. Consider a graph $\mathcal{G}$. The degree of a node $u \in \mathcal{V}$ corresponds to the number of nodes neighbouring u in $\mathcal{G}$. In particular, $deg(u) = |\{v \in \mathcal{V} \mid (u, v) \in \mathcal{E}\}|$

Definition 2.4. The adjacency matrix of a graph $\mathcal{G}$ of order n corresponds to a matrix $A \in \mathcal{M}_{n\times n}(\mathbb{C})$ for which

$$\forall (i,j) \in [n]^2, A_{i,j} = \begin{cases} 1 : (i,j) \in \mathcal{E} \\ 0 : \text{ otherwise} \end{cases}$$

In other words, for each pair of nodes, it marks the existence of an edge between them. Note that in the case of an undirected graph, A is symmetric.

Definition 2.5. Types of traversals.

- A **walk** of length ℓ in a graph $\mathcal{G}$ corresponds to a sequence of vertices (and implicitly the edges between them), $v_0, \ldots, v_\ell$ with $v_i \in \mathcal{E}$. Note that ℓ corresponds to the number of edges present in the walk.
- A **closed walk** of length ℓ in a graph $\mathcal{G}$ corresponds to a sequence of vertices $v_0, \ldots, v_\ell$ with $v_i \in \mathcal{E}$ for which $v_0 = v_\ell$. Often times, the last node in the sequence is omitted.
- A **cycle** of length ℓ in a graph $\mathcal{G}$ corresponds to a closed walk with no repetitions of vertices except that the starting and ending vertex. This means that apart from the first/last vertex, every vertex appears only once.

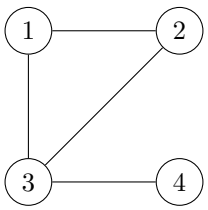

Figure 1: Graph of order 4 with one cycle of length 3

In figure 2.3 we can appreciate a graph $\mathcal{G}$ of order 4 defined by

$$\mathcal{V} = \{1,2,3,4\}$$
$$\mathcal{E} = \{(1,2),(1,3),(2,3),(3,4)\}$$

Note that this graph contains the cycle $1,2,3$ of length three and the closed walk $4,3,2,1,3,4$ of length five. Furthermore the adjacency matrix of this graph is,

$$A = \begin{pmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}$$

Proposition 2.1. Let $\mathcal{G}$ be a graph of order n of adjacency matrix $A \in \mathcal{M}_{n\times n}(\mathbb{C})$. Let $0 \leq i,j \leq n$. The number of walks of length ℓ in $\mathcal{G}$ that start on vertex i and end on vertex j corresponds to the (i,j)-th entry of A^ℓ.

Proof. We will proceed by induction on the size of the walk.

Base case. If $\ell = 1$, then we can clearly see that the number of ways to go from i to j corresponds to $A_{i,j}$.

Induction. Suppose that $(A^\ell)_{i,j}$ counts the number of walks of length ℓ from i to j. Consider $A^{\ell+1} = A \cdot A^\ell$, then

$$A^{\ell+1}_{i,j} = \sum_{k=1}^{n} (A^\ell)_{i,k} \cdot A_{k,j}$$

By induction hypothesis, $(A^\ell)_{i,k}$ corresponds to the number of walks of length ℓ from i to k. Furthermore, $(A^\ell)_{i,k}$ only contributes to the final result if there is an edge from k to j. Indeed $A_{k,j} = 1 \iff (k,j) \in \mathcal{E}$. Hence, since we iterate through all nodes in $\mathcal{G}$, the final result $(A^{\ell+1})_{i,j}$ corresponds to all walks of length $\ell + 1$ from i to j. □

Definition 2.6. Complete Graphs.

A graph $\mathcal{G}$ of order n is complete if all nodes are of degree $n - 1$. Such a graph will be denoted as K_n. An example of K_4 can be seen in figure 2.3

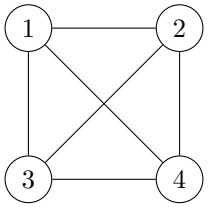

Figure 2: K_4 - Complete graph of order 4

Proposition 2.2. Let $n > 1$. The number of unique cycles of length $3 \leq \ell \leq n$ in K_n is $\binom{n}{\ell} \cdot \frac{(\ell-1)!}{2}$

Proof. Indeed, it suffices to choose ℓ out of n vertices to compose the cycle for which there are $\binom{n}{\ell}$ choices. Furthermore, once the ℓ vertices have been chosen, there are $\ell!$ possible permutations for the ℓ nodes.

Notice that we count all cycles 2ℓ times more than we should. Indeed, suppose $(u_1, \ldots, u_\ell)$ represents a cycle. All possible rotations and inversions of $(u_1, \ldots, u_\ell)$ are considered the same cycle as the graph is undirected. Since there are ℓ possible rotations and two possible directions (forwards and backwards) we get the expression

$$\binom{n}{\ell} \cdot \frac{\ell!}{2l} = \binom{n}{\ell} \cdot \frac{(\ell-1)!}{2} = \frac{n!}{2\ell(n-\ell)!}$$

□

Definition 2.7. n-Partite Graphs.

A graph $\mathcal{G}$ is called n-partite graph if its vertex set $\mathcal{V}$ is partitioned into disjoint n sets $\mathcal{V}_1, \ldots, \mathcal{V}_n$ such that for all $1 \leq i \leq n$, if $u \in \mathcal{V}_i$, then $\nexists v \in \mathcal{V}_i$ such that $(u, v) \in \mathcal{E}$. In other words, no edge connects vertices within the same subset.

Definition 2.8. Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ be an n-partite graph with $\mathcal{V} := \bigsqcup_{i=1}^{n} \mathcal{V}_i$ and $m_i := |\mathcal{V}_i|$. We will call $\mathcal{G}$ a complete n-partite graph, denoted $K_{m_1 \times \ldots \times m_n}$, if for all $i \in [n]$ and $u \in \mathcal{V}_i$, $deg(u) = \prod_{j \neq i} m_j$. In other words, every node of $\mathcal{G}$ is connected to every vertex of all other partitions.

In figure 3 we can appreciate a complete tripartite graph $K_{2\times2\times2}$.

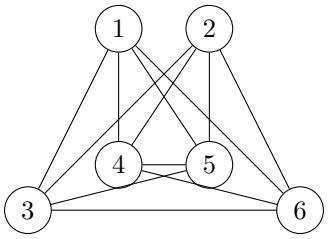

Figure 3: $K_{2\times2\times2}$ - Complete tripartite graph

Proposition 2.3. The total number of closed walks of length $\ell \geq 3$ in a complete tripartite graph $K_{n\times n\times n}$ is $(2n)^\ell + 2(-n)^\ell$

Proof. Consider $A \in \mathcal{M}_{3n\times3n}(\mathbb{C})$ the adjacency matrix of $K_{n\times n\times n}$.

$$A := \begin{pmatrix} 0_{n\times n} & 1_{n\times n} & 1_{n\times n} \\ 1_{n\times n} & 0_{n\times n} & 1_{n\times n} \\ 1_{n\times n} & 1_{n\times n} & 0_{n\times n} \end{pmatrix}$$

A simple way of finding the eigenvalues of A is through the Kronecker product [10].

Definition 2.9. Let

$$P = \begin{pmatrix} p_{1,1} & \cdots & p_{1,m} \\ \vdots & \ddots & \vdots \\ p_{1,n} & \cdots & p_{n,m} \end{pmatrix} \in \mathcal{M}_{n\times m}(\mathbb{C})$$

and $Q \in \mathcal{M}_{n'\times m'}(\mathbb{C})$. The Kronecker product of P and Q noted $P \otimes Q$ is defined as the block matrix

$$P \otimes Q := \begin{pmatrix} p_{1,1}Q & \cdots & p_{1,n}Q \\ \vdots & \ddots & \vdots \\ p_{1,n}Q & \cdots & p_{n,n}Q \end{pmatrix}$$

In our particular case $A = B \otimes 1_{n\times n}$ with $B := \begin{pmatrix} 0 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 0 \end{pmatrix}$

Proposition 2.4 (4.2.12 in [10])**.** Let $P \in \mathcal{M}_{n\times n}(\mathbb{C}), Q \in \mathcal{M}_{n'\times n'}(\mathbb{C})$. Denote $\sigma(\cdot)$ the spectre of a given matrix. Let $\lambda \in \sigma(P)$ and $\mu \in \sigma(Q)$, then $\lambda \cdot \mu \in \sigma(P \otimes Q)$

We know that the eigenvalues of B are 2 with multiplicity 1 and -1 with multiplicity 2. Furthermore, the eigenvalues of $1_{n\times n}$ are n with multiplicity 1 and 0 with multiplicity $n-1$. Therefore, applying the previous proposition, combining the specters of B and $1_{n\times n}$ we get that the eigenvalues of A are $2n$ with multiplicity 1, $-n$ with multiplicity 2 and 0 with multiplicity $(3n-3)$. Thus, by proposition 2.1, the total amount of closed walks of length ℓ in $K_{n\times n\times n}$ is $(2n)^\ell + 2(-n)^\ell$. □

2.4. Tensor Graphs

As previously introduced, the main focus of this report will be on the algebraic study of tensor graphs. Given the previously introduced formalisms we will now describe how such an object is constructed.

Definition 2.10 (Graph associated to a 3-tensor)**.** Given $C \in (U \otimes V \otimes W)^*$, we will note $\mathcal{G}(\mathcal{C}) := (\mathcal{V}_\mathcal{C}, \mathcal{E}_\mathcal{C})$, the tri-partite graph associated to $\mathcal{C}$. The set of vertices is defined as the disjoint union,

$$\mathcal{V}_\mathcal{C} := \mathbb{P}(U) \sqcup \mathbb{P}(V) \sqcup \mathbb{P}(W)$$

Likewise, the set of edges is defined as,

$$\begin{aligned} \mathcal{E}_\mathcal{C} := &\{(\hat{u}, \hat{v}) \in \mathbb{P}(U) \times \mathbb{P}(V) \mid \mathcal{C}(\hat{u}, \hat{v}, -) = 0\} \\ &\cup \{(\hat{u}, \hat{w}) \in \mathbb{P}(U) \times \mathbb{P}(W) \mid \mathcal{C}(\hat{u}, -, \hat{w}) = 0\} \\ &\cup \{(\hat{v}, \hat{w}) \in \mathbb{P}(V) \times \mathbb{P}(W) \mid \mathcal{C}(-, \hat{v}, \hat{w}) = 0\} \end{aligned}$$

Definition 2.11 (Triangle)**.** Let $\mathcal{C} : U \times V \times W \to \mathbb{F}_q$ be a 3-tensor, $(\hat{u}, \hat{v}, \hat{w}) \in \mathbb{T}(U, V, W) := \mathbb{P}(U) \times \mathbb{P}(V) \times \mathbb{P}(W)$ is a triangle for $\mathcal{C}$ iff

$$\mathcal{C}(\hat{u}, \hat{v}, -) = 0, \qquad \mathcal{C}(\hat{u}, -, \hat{w}) = 0, \qquad \mathcal{C}(-, \hat{v}, \hat{w}) = 0$$

In other words, $(\hat{u}, \hat{v}, \hat{w})$ is a 3-cycle in $\mathcal{G}(\mathcal{C})$. We will note $\mathbb{T}(\mathcal{C})$ the set of triangles in $\mathcal{G}(\mathcal{C})$.

In figure 4 we can see an example of a 3-tensor graph represented by

$$\left[\begin{pmatrix} 2 & 0 & 3 \\ 4 & 0 & 1 \\ 3 & 0 & 4 \end{pmatrix}, \begin{pmatrix} 2 & 1 & 4 \\ 2 & 2 & 4 \\ 4 & 2 & 0 \end{pmatrix}, \begin{pmatrix} 2 & 0 & 4 \\ 4 & 0 & 3 \\ 4 & 4 & 0 \end{pmatrix} \right]$$

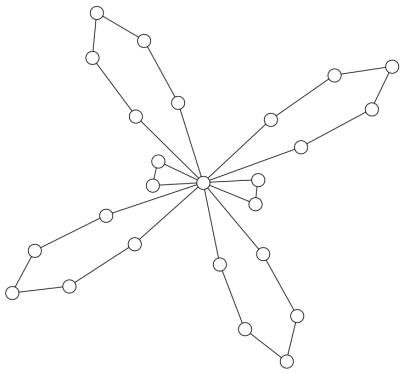

Figure 4: Unlabeled graph of a 3-tensor of dimensions $(3,3,3)$ in $\mathbb{F}_5$. Nodes of null-degree hidden.

We notice too that it posseses two triangles for which the vectors,

$$\begin{aligned}(u_1,v_1,w_1) &= \left[\begin{pmatrix}1 & 0 & 0\end{pmatrix}, \quad \begin{pmatrix}3 & 1 & 0\end{pmatrix}, \quad \begin{pmatrix}0 & 1 & 0\end{pmatrix}\right] \\ (u_2,v_2,w_2) &= \left[\begin{pmatrix}0 & 0 & 1\end{pmatrix}, \quad \begin{pmatrix}3 & 1 & 0\end{pmatrix}, \quad \begin{pmatrix}3 & 2 & 1\end{pmatrix}\right]\end{aligned}$$

are representatives.

Remark that the trilinearity of a tensor $\mathcal{C} \in (U \otimes V \otimes W)^*$ allows for a direct mapping of the nodes and edges in $\mathcal{G}(\mathcal{C})$ to the nodes and edges of any other graph for any other tensor in the orbit of $\mathcal{C}$. In particular, let $\mathcal{D} = \mathcal{C} \circ (A, B, C) \in (U \otimes V \otimes W)^*$. Then,

$$\begin{aligned}\mathcal{E}_{\mathcal{D}} = &\{((Au, Bv) \mid (u,v) \in \mathcal{E}_{\mathcal{C}} \cap \mathbb{P}(U) \times \mathbb{P}(V)\} \\ &\cup \{((Au, Cw) \mid (u,w) \in \mathcal{E}_{\mathcal{C}} \cap \mathbb{P}(U) \times \mathbb{P}(W)\} \\ &\cup \{((Bv, Cw) \mid (v,w) \in \mathcal{E}_{\mathcal{C}} \cap \mathbb{P}(V) \times \mathbb{P}(W)\}\end{aligned}$$

In simple terms, this tells us that for all tensors in the orbit of $\mathcal{C}$, their respective tensor graphs share the same graph invariants as $\mathcal{G}(\mathcal{C})$.

Example 2.3 (Isomorphism invariance of projective triangles)**.** Consider the following tensor in $\mathbb{F}_5$.

$$\mathcal{T} = \left[\begin{pmatrix}2 & 4 & 0\\ 1 & 3 & 1\\ 4 & 4 & 1\end{pmatrix}, \begin{pmatrix}1 & 4 & 4\\ 1 & 3 & 0\\ 3 & 0 & 1\end{pmatrix}, \begin{pmatrix}0 & 4 & 4\\ 1 & 3 & 1\\ 3 & 3 & 1\end{pmatrix}\right]$$

Define,

$$A := \begin{pmatrix}1 & 3 & 0\\ 2 & 1 & 3\\ 0 & 3 & 3\end{pmatrix}, \quad B := \begin{pmatrix}3 & 1 & 4\\ 4 & 0 & 0\\ 4 & 0 & 1\end{pmatrix}, \quad C := \begin{pmatrix}0 & 1 & 0\\ 4 & 0 & 0\\ 2 & 1 & 1\end{pmatrix} \in GL(\mathbb{F}_5, 3)$$

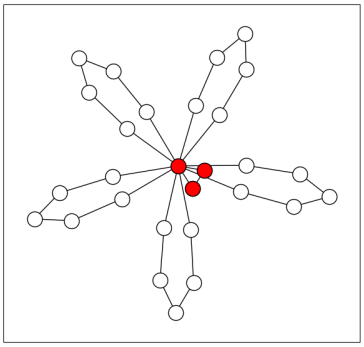

Figure 5: Tensor graph of $\mathcal{T}$ and $\mathcal{T} \circ (A, B, C)$

and

$$\mathcal{T}' := \mathcal{T} \circ (A, B, C) = \left[\begin{pmatrix} 2 & 4 & 0 \\ 4 & 2 & 3 \\ 3 & 1 & 0 \end{pmatrix}, \begin{pmatrix} 3 & 0 & 0 \\ 4 & 3 & 1 \\ 4 & 3 & 1 \end{pmatrix}, \begin{pmatrix} 1 & 3 & 3 \\ 4 & 2 & 4 \\ 4 & 2 & 2 \end{pmatrix} \right]$$

It can be easily verified that the triple

$$(\hat{u}, \hat{v}, \hat{w}) = [(4, 4, 1), (3, 1, 0), (2, 1, 1)]$$

is a projective triangle in $\mathcal{G}(\mathcal{T})$. Similarly, the triple

$$(u', v', w') = [(2, 0, 1), (1, 0, 1), (4, 2, 1)]$$

is a triangle in $\mathcal{G}(\mathcal{T}')$. Finally, one can check that $\lambda_1 u = Au', \lambda_2 v = Bv'$, and $\lambda_3 w = w'$ for $\lambda_i \in \mathbb{F}_5$. Their corresponding tensor graph can be seen in figure 5.

2.5. Signature Schemes and Parameter sizes

We will finish this preliminary section by instantiating the parameters of the 3-TIP to concrete, cryptographically-relevant sizes. ALTEQ parameter specifications [3] define a single field size $q = 2^{32} - 5$ and three levels of dimension sizes for which $n = m = k$: 13, 20, and 25. Similarly, MEDS parameter specifications [7] consider equal dimension sizes $n = m = k$ for three different security levels. MEDS field size, however, changes as a function of the security level. This information can be seen in table 1.

Security Level	ALTEQ		MEDS	
	q	n	q	n
1	$2^{32}-5$	13	4093	14
2	$2^{32}-5$	20	4093	22
3	$2^{32}-5$	25	2039	30

Table 1: Recommended parameters for ALTEQ and MEDS signature schemes.

3. On the size of tensor graphs

We begin this report by asking a simple, but crucial question: How large can a tensor graph realistically become? We first derive closed-form expressions for the number of vertices and edges that appear when a 3-tensor is mapped to its tripartite graph, and we then justify that the expected edge count grows much more slowly in comparison. These formulas immediately imply a sparsity gap that widens with either the dimension or the field size. We continue by specializing these asymptotic results to the MEDS signature parameters, highlighting that $|V| \simeq q^{3n}$ yet $|E| \simeq q^n$, and we visualize this divergence with a logarithmic plot. These results motivate later choices that exploit the graph's sparsity and justify the limitations of a later implementation.

Proposition 3.1. The number of nodes in a tensor graph is

$$|\mathcal{V}_{\mathcal{C}}| = \frac{(q^n-1)(q^m-1)(q^k-1)}{(q-1)^3} = O(q^{n+m+k})$$

Proof. We will start by considering U, the argument being equivalent for V and W. There are $q^n - 1$ non-zero elements in U as U is an n-dimensional vector space over $\mathbb{F}_q$. Likewise, for each one-dimensional vector space $\hat{u}$ of U, there are $q-1$ non-zero elements since

$$\hat{u} := \{a \cdot u \mid a \in \mathbb{F}_q\}$$

Therefore, there are $\frac{q^n-1}{q-1}$ elements in $\mathbb{P}(U)$. We conclude by recalling that

$$\mathcal{V}_{\mathcal{C}} = \mathbb{P}(U) \times \mathbb{P}(V) \times \mathbb{P}(W)$$

□

Proposition 3.2. Given a uniformly random tensor $\mathcal{C}$, the probability that $(u, v) \in U \times V$ form an edge is

$$\Pr_{\mathcal{C} \sim (U \otimes V \otimes W)^*}[(\hat{u}, \hat{v}) \in \mathcal{E}_{\mathcal{C}}] = \frac{1}{q^k}$$

Proof. Consider $(u, v) \in U \times V$ and a uniformly random tensor $\mathcal{C} \in (U \otimes V \otimes W)^*$. Recall that $(\hat{u}, \hat{v}) \in \mathcal{E}_{\mathcal{C}} \iff \mathcal{C}(u, v, -) = 0$. The application $\mathcal{C}_{u,v} : w \mapsto \mathcal{C}(u, v, w)$ is a uniformly random linear form in W^*. Thus, $\mathcal{C}_{u,v}$ is the null linear form with probability $1/q^k$. A similar argument follows for edges in $\mathbb{P}(U) \times \mathbb{P}(W)$ and $\mathbb{P}(V) \times \mathbb{P}(W)$. □

Corollary 3.1. The expected number of edges in a tensor graph is

$$\begin{aligned}\mathbb{E}_{\mathcal{C}\sim(\mathcal{U}\otimes\mathcal{V}\otimes\mathcal{W})^*}[|\mathcal{E}_\mathcal{C}|] &= \frac{(q^n-1)(q^m-1)}{(q-1)^2q^k} + \frac{(q^n-1)(q^k-1)}{(q-1)^2q^m} + \frac{(q^m-1)(q^k-1)}{(q-1)^2q^n} \\ &= O\big(q^{n+m+k}(q^{-2n}+q^{-2m}+q^{-2k})\big)\end{aligned}$$

Proof. Recalling the proof for prop. 3.1 we remark that there are

$$\frac{(q^n-1)(q^m-1)}{(q-1)^2}$$

elements in $\mathbb{P}(U)\times\mathbb{P}(V)$. For each pair of points, the probability of them forming an edge is $1/q^k$ by 3.2. Therefore, in expectancy, there are only

$$\frac{(q^n-1)(q^m-1)}{(q-1)^2q^k}$$

edges from $\mathbb{P}(U)\times\mathbb{P}(V)$. A similar argument follows for edges in $\mathbb{P}(U)\times\mathbb{P}(W)$ and $\mathbb{P}(V)\times\mathbb{P}(W)$. Therefore, by linearity of expectation, the expected total number of edges in $\mathcal{G}(\mathcal{C})$ is

$$\frac{(q^n-1)(q^m-1)}{(q-1)^2q^k} + \frac{(q^n-1)(q^k-1)}{(q-1)^2q^m} + \frac{(q^m-1)(q^k-1)}{(q-1)^2q^n}$$

□

3.1. MEDS graphs

Our analysis on the growth of tensor graphs is vastly simplified by considering that in practice, all vector-space dimensions are equal (see table 1). In such a case, in expectancy, $|\mathcal{V}_\mathcal{C}| = O(q^{3n})$ and $|\mathcal{E}_\mathcal{C}| = O(q^n)$ with $n = m = k$. These shorthand formulas emphasize that the number of nodes as a function of the dimension increases exponentially faster than the number of edges. Equivalently, in the case of constant dimension size and variable field size, the number of nodes grows cubically faster than the number of edges. The growth of such tensor graphs as a function of the security level can be seen in figure 6.

4. On the shape of tensor graphs

Having measured how big tensor graphs are, in this section we will turn to what they look like by analyzing the small invariant sub-structures in such sparse graphs. We begin by looking at triangles, giving an alternative proof to [14] that a random tensor graph contains only $\Theta(1/q)$ of them on average. We then widen the lens to closed walks: first exhaustively classifying all six configurations of length 4 and reporting experimental counts, and next giving length-independent estimations that again decay as $\Theta(1/q)$ for any fixed length $\ell \geq 3$. Collectively, these results paint a geometric portrait of tensor graphs that informs both the isomorphism-finding algorithm in Section 5 and the implementation choices in Section 6.

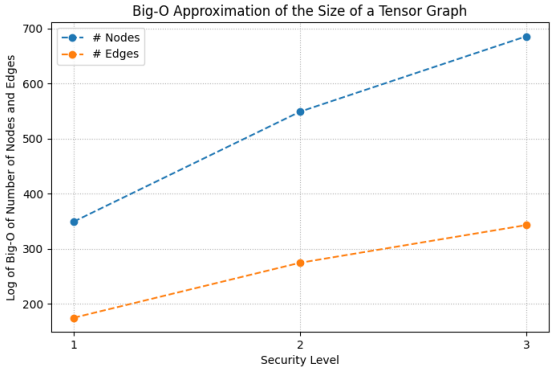

Figure 6: Logarithmic upper bound on the sizes of graph sizes per security level

4.1. Triangles

We will now recall the expected number of triangles in a random tensor graph as proven by Ran and Samardjiska [14]. One must highlight that their proof relies heavily on the conjecture that the evaluation of a random tensor at two different triplets of projective points is independent.

Proposition 4.1. Let U,V,W be n, m, k-dimensional vector spaces over $\mathbb{F}_q$,

$$\mathbb{E}_{\mathcal{C}\sim(U\otimes V\otimes W)^*}[|\mathbb{T}(\mathcal{C})|] = \Theta(1/q)$$

Proof. We will start by finding the probability that an arbitrary triple forms a triangle in the tensor graph. Let $(\hat{u}, \hat{v}, \hat{w}) \in \mathbb{P}(U) \times \mathbb{P}(V) \times \mathbb{P}(W)$ where u, v, w are non-zero representatives. Extend u, v and w into the bases

$$U_B = \langle u, u_2, \ldots, u_n\rangle, \qquad V_B = \langle v, v_2, \ldots, v_n\rangle, \qquad W_B = \langle w, w_2, \ldots, w_n\rangle$$

Consider the tensor $\mathcal{C}'$ in this basis. In other word, consider the tensor $\mathcal{C}'$ such that

$$\mathcal{C} = \mathcal{C}' \circ (U_B, V_B, W_B)$$

Notice that (u, v, w) is a triangle in $\mathcal{C}$ iff (e_1^U, e_1^V, e_1^W) is a triangle in C'. This implies that for all $(u', v', w') \in \mathbb{P}(U) \times \mathbb{P}(V) \times \mathbb{P}(W)$,

$$\mathcal{C}'(e_1^U, e_1^V, w') = \mathcal{C}'(e_1^U, v', e_1^W) = \mathcal{C}'(u', e_1^V, e_1^W) = 0$$

Or simply, for $1 \leq i \leq n$, $1 \leq j \leq m$, $1 \leq l \leq k$,

$$\mathcal{C}'(e_1^U, e_1^V, e_l^W) = \mathcal{C}'(e_1^U, e_j^V, e_1^W) = \mathcal{C}'(e_i^U, e_1^V, e_1^W) = 0$$

Consider $\mathcal{C}$ as a uniform trivariate random variable in the set $\mathbb{F}_q$. Denote E_W, E_V, E_U the events for which $(e_1^U, e_1^V), (e_1^U, e_1^W)$, and (e_1^U, e_1^W) respectively,

form an edge. Formally,

$$\begin{cases} E_U = \bigcap_{i=1}^{n} (\mathcal{C}'(e_i^U, e_1^V, e_1^W) = 0) \\ E_V = \bigcap_{j=1}^{m} (\mathcal{C}'(e_1^U, e_j^V, e_1^W) = 0) \\ E_W = \bigcap_{l=1}^{k} (\mathcal{C}'(e_1^U, e_1^V, e_l^W) = 0) \end{cases}$$

Therefore, by independence and sigma additivity,

$$\begin{aligned} \Pr_{\mathcal{C} \sim (U \otimes V \otimes W)^*} [(u, v, w) \in \mathbb{T}(\mathcal{C})] &= \Pr_{\mathcal{C} \sim (U \otimes V \otimes W)^*} [E_U \cup E_V \cup E_W] \\ &= q^{-|E_U| - |E_V| - |E_W| + |E_U \cap E_V| + |E_U \cap E_W| + |E_V \cap E_W| - |E_U \cap E_V \cap E_W|} \\ &= q^{-n-m-k+2} \end{aligned}$$

Remark that this is the case because the event $\mathcal{C}'(e_1^U, e_1^V, e_1^W)$ appears in all E_U, E_V, E_W thus it must be excluded twice. Finally, by linearity of expectation,

$$\begin{aligned} \mathbb{E}[|\mathbb{T}(\mathcal{C})|]_{\mathcal{C} \sim (U \otimes V \otimes W)^*} &= |\mathbb{P}(U) \times \mathbb{P}(V) \times \mathbb{P}(W)| \cdot q^{-(n+m+k-2)} \\ &= \frac{(q^n - 1)(q^m - 1)(q^k - 1)}{(q-1)^3 q^{n+m+k-2}} \\ &= \Theta(1/q) \end{aligned}$$

□

4.2. Closed walks of length 4

Motivated by the frequency of triangles in random tensor graphs, we take a first step toward generalizing these results by characterizing closed walks of length four. This section presents both the theoretical framework and supporting experimental findings. Let $n = m = k$. Let $\mathcal{C} \in (U \otimes V \otimes W)^*$. The problem of finding all closed walks of length $\ell = 4$ in $\mathcal{G}(\mathcal{C})$ can be modeled as a system of quadratic equations in the ring:

$$R = \mathbb{F}_q[x_1, \ldots, x_{6n}]$$

Denote,

$$\begin{aligned} u &:= [x_1, \ldots, x_n] & u' &:= [x_{n+1}, \ldots, x_{2n}] \\ v &:= [x_{2n+1}, \ldots, x_{3n}] & v' &:= [x_{3n+1}, \ldots, x_{4n}] \\ w &:= [x_{4n+1}, \ldots, x_{5n}] & w' &:= [x_{5n+1}, \ldots, x_{6n}] \end{aligned}$$

To find a closed walk of length four it suffices to solve the following systems of equations.

- **Type A** Closed walks with two nodes in $\mathbb{P}(U)$, and two nodes in $\mathbb{P}(V)$.

$$\begin{cases} \mathcal{C}(u, v, e_{i_1}^W) = 0 \\ \mathcal{C}(u', v, e_{i_2}^W) = 0 \\ \mathcal{C}(u', v', e_{i_3}^W) = 0 \\ \mathcal{C}(u, v', e_{i_4}^W) = 0 \end{cases} \qquad \forall i_j \in [n]$$

- **Type B** Two nodes in $\mathbb{P}(U)$, and two nodes in $\mathbb{P}(W)$.

$$\begin{cases} \mathcal{C}(u, e_{i_1}^V, w) & = 0 \\ \mathcal{C}(u', e_{i_2}^V, w) & = 0 \\ \mathcal{C}(u', e_{i_3}^V, w') & = 0 \\ \mathcal{C}(u, e_{i_4}^V, w') & = 0 \end{cases} \qquad \forall i_j \in [n]$$

- **Type C** Two nodes in $\mathbb{P}(V)$, and two nodes in $\mathbb{P}(W)$

$$\begin{cases} \mathcal{C}(e_{i_1}^U, v, w) & = 0 \\ \mathcal{C}(e_{i_2}^U, v', w) & = 0 \\ \mathcal{C}(e_{i_3}^U, v', w') & = 0 \\ \mathcal{C}(e_{i_4}^U, v, w') & = 0 \end{cases} \qquad \forall i_j \in [n]$$

- **Type D** Two node in $\mathbb{P}(U)$, one in $\mathbb{P}(V)$, and one in $\mathbb{P}(W)$

$$\begin{cases} \mathcal{C}(u, v, e_{i_1}^W) & = 0 \\ \mathcal{C}(u', v, e_{i_2}^W) & = 0 \\ \mathcal{C}(u', e_{i_3}^V, w) & = 0 \\ \mathcal{C}(u, e_{i_4}^V, w) & = 0 \end{cases} \qquad \forall i_j \in [n]$$

- **Type E** One node in $\mathbb{P}(U)$, two in $\mathbb{P}(V)$, and one in $\mathbb{P}(W)$

$$\begin{cases} \mathcal{C}(u, v, e_{i_1}^W) & = 0 \\ \mathcal{C}(u, v', e_{i_2}^W) & = 0 \\ \mathcal{C}(e_{i_3}^U, v', w) & = 0 \\ \mathcal{C}(e_{i_4}^U, v, w) & = 0 \end{cases} \qquad \forall i_j \in [n]$$

- **Type F** One node in $\mathbb{P}(U)$, one in $\mathbb{P}(V)$, and two in $\mathbb{P}(W)$

$$\begin{cases} \mathcal{C}(u, e_{i_1}^V, w) & = 0 \\ \mathcal{C}(u, e_{i_2}^V, w') & = 0 \\ \mathcal{C}(e_{i_3}^U, v, w') & = 0 \\ \mathcal{C}(e_{i_4}^U, v, w) & = 0 \end{cases} \qquad \forall i_j \in [n]$$

Without restricting the coordinates of the unknown vectors, each of the six systems of equations consists of $4n$ unknowns and $4n$ equations. Since each type of walk is independent of each other, all six systems have to be solved independently. Therefore, to find all possible closed 4-walks we end up solving for $24n$ unknowns spread through six systems of $4n$ quadratic equations.

All these different types of walks can be seen in Figure 4.2 assuming $u, u' \in \mathbb{P}(U), v, v' \in \mathbb{P}(V)$, and $w, w' \in \mathbb{P}(W)$. Note that we may have $u = u'$, $v = v'$,

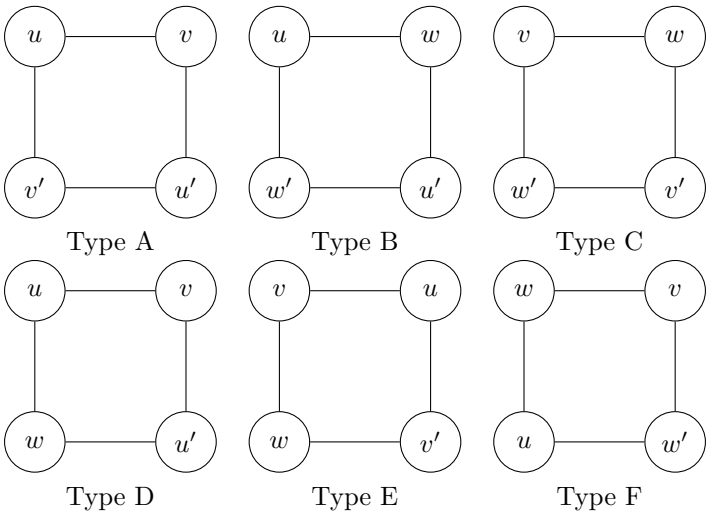

Figure 7: Visualization of types of closed walks of length 4.

or $w = w'$. Some extra filtering can be done to discard walks where $u = u'$, $v = v'$, and $w = w'$, thus isolating genuine cycles rather than general closed walks.

Remark that in practice, one may need to impose different "starting" coordinates for unknowns belonging to the same projective space so that the dimension of the ideal whose variety we compute (i.e., the system we solve) is zero, ensuring isolated solutions. For example, for u and u' we might set $u_0 = x_0 = 1$, $u_1 = x_1 = 0$, and $u'_0 = x_{n+1} = 0$, $u'_1 = x_{n+2} = 1$. This is done to break the residual projective symmetry, forcing u and u' to be distinct points in $\mathbb{P}(U)$ and guaranteeing that the ideal is zero-dimensional (so that actual solutions can be found).

Some experimental results using this coordinate-fixing strategy to find closed walks over 10 000 random tensors per parameter are summarized in table 2. The average running time and memory usage required to find all closed walks of all types for a single random 3-tensor, under each parameter configuration, are shown to the right. These tests were conducted on a system with an Intel i5-1155G7 (8 cores) processor and 16GiB of RAM.

Regarding the implementation[1], as previously discussed, every vertex is put into a normalized representative form so that the Gröbner ideal is zero-dimensional. For the two-partition patterns (A, B and C) each of the four vertices must satisfy two scalar constraints, e.g. for type A we fix $u_0 = 1, u_1 = 0$ and $u'_0 = 0,\ u'_1 = 1$. Forcing the first non-zero coordinate to 1 selects roughly a $1/q$ fraction of all points; imposing one further coordinate equal to 0 multiplies

[1] see `https://github.com/david-pulido/3TI_Graph_Visualizer`

Parameters				Walk types						Avg. resources/tensor	
n	m	k	q	A	B	C	D	E	F	time (s)	memory (mB)
3	3	3	7	0	0	0	2	3	6	0.70	196.18
3	3	3	11	0	0	0	1	3	1	0.70	196.17
3	3	3	17	0	0	0	0	0	1	0.69	196.19
4	4	4	7	0	0	0	2	6	2	0.71	196.45
4	4	4	11	0	0	0	0	2	1	0.90	196.45
4	4	4	17	0	0	0	0	0	0	0.73	196.45
5	5	5	7	0	0	0	5	4	3	1.03	198.15
5	5	5	11	0	0	0	1	0	1	0.97	198.18
5	5	5	17	0	0	0	0	0	0	0.96	198.24

Table 2: Counts of each walk type found across 10 000 runs for each parameter combination

this by another $1/q$. Therefore, account all four vertices, the solver draws its results from only $(1/q)^8$-fraction of all quadruples for patterns $A - C$.

Similarly, the mixed-partition patterns D, E, and F are pinned almost as tightly, but not quite. For instance, in the case of type D walks, each U-vertex again has two fixed coordinates, whereas the two vertices that lie in the other two partitions (V and W) are subject only to the single requirement that their first coordinate equals 1. Counting the constraints vertex by vertex, the overall thinning factor for D, E, F ends up being roughly q^{-6}.

An alternative method to increase the proportion of closed walks found, not implemented due to time and resource limitations, would consist of systematically alternating the positions and values of the normalization and zero constraints applied to the projective variables. This would allow the solver to explore a larger subset of the projective space. However, note that this strategy would still not cover all possible closed walks, as the imposed zero constraints remain highly restrictive and exclude a fraction of valid tuples.

Turning to the experimental results, table 2 confirms four trends. First, the solver never encounters 4-cycles of types A, B or C, no matter the dimension or field size. Second, the mixed-partition patterns D, E, and F do appear, but only in small quantities. Third, for a fixed value of q, the total number of closed walks appears relatively stable as the dimension varies. Fourth, run-time stays under one second and memory flat-lines around 196-198 MB across all experiments, indicating that the Gröbner bases the program solves have essentially constant size once n, m, and k are fixed.

The discrepancy in the number of A, B, C walks and D, E, F walks can mainly be explained due to the more relaxed coordinate pinning for $D - F$ walks. As it was previously introduced, types A, B, C pin eight homogeneous coordinates, thinning the candidate space by q^{-8} and making the system over-determined (so no solutions in practice), whereas types D, E, F pin only six (thinning by q^{-6}), leaving a lightly over-determined system that still admits a few isolated solutions in $\mathbb{F}_q$ for small q, exactly matching the handful of D, E, F

walks seen at $q = 7$ and their rapid disappearance as q increases.

4.3. Closed walks of arbitrary length

Building on the triangle and length-four analysis, this subsection extends the study to closed walks of any length $\ell \geq 3$. Rather than attempting the combinatorially challenging task of enumerating every possible cycle, we exploit the fact that every cycle is embedded in at least one closed walk of the same length. By upper-bounding cycles with closed walks we obtain a manageable counting problem and prove an ℓ-independent, dimension-independent expectation for the number of such walks in a random tensor graph.

Proposition 4.2. Let $\ell \geq 3$ and $\mathcal{C} \in (U \otimes V \otimes W)^*$ with $n = m = k$. The probability that ℓ nodes form a closed walk in $\mathcal{G}(\mathcal{C})$ as $q \to \infty$ is in

$$q^{-\ell n - \ell + 1}$$

Proof. Consider the tensor $\mathcal{C}$ as a uniformly random element in $\mathbb{F}_q^{n^3}$. Given ℓ points $x_1, \ldots, x_\ell \in \mathbb{P}(U) \cup \mathbb{P}(V) \cup \mathbb{P}(W)$, the existence of an edge between x_i and x_{i+1} imposes n constraints on the n^3 coefficients of $\mathcal{C}$. For example, for the pair $(\hat{u}, \hat{w}) \in \mathbb{P}(U) \times \mathbb{P}(W)$, we require that $\forall i \in [n], \mathcal{C}(u, v, e_i^W) = 0$. Accounting for all ℓ edges, we then obtain a $\ell n \times n^3$ constraint matrix on the coefficients of $\mathcal{C}$.

However, each x_i represents a point in projective space $\mathbb{P}(U)$, $\mathbb{P}(V)$ or $\mathbb{P}(W)$, meaning there is a scalar degree of freedom in choosing a representative vector. Because there are ℓ such nodes, we have altogether ℓ scalar rescalings (one for each node) that do not change the geometry of the potential walk. Nonetheless, if we scale all ℓ points by the same factor, we obtain a global redundant rescaling (it amounts to doing nothing overall, since all edges remain the same). Consequently, we gain $\ell - 1$ degrees of freedom.

Heuristically, we assume that the constraint matrix is of full rank as $q \to \infty$. Indeed, we can argue that the constraint matrix can be modeled as a random matrix for an arbitrary choice of vertices in the walk, so for big enough q, the constrain matrix will be of rank $n - d$ with probability $1/q^d$ as $q \to \infty$ [6], i.e. full rank with high probability. Therefore, the total number of independent linear constraints is $\ell n - \ell + 1$ with high probability. This implies that, out of the n^3 dimensions of the tensor space, $\ell n - \ell + 1$ of them are constrained by the edge conditioning, accounting for projectivity. Finally, the probability that ℓ nodes form a closed walk in $\mathcal{G}(\mathcal{C})$ as $q \to \infty$ is,

$$\frac{q^{n^3 - \ell n + \ell - 1}}{q^{n^3}} = \frac{1}{q^{\ell n - \ell + 1}}$$

□

Proposition 4.3 (Closed walks of length $\ell \geq 3$)**.** Let $n = m = k$ and $\mathcal{C} \in (U \otimes V \otimes W)^*$ a uniformly random tensor. Consider $M_\ell(\mathcal{C})$ the number of closed walks of length $\ell \geq 3$ in the tensor graph $\mathcal{G}(\mathcal{C})$. Then,

$$\mathbb{E}_{\mathcal{C} \sim (U \otimes V \otimes W)^*}[M_\ell(\mathcal{C})] = \Theta(1/q)$$

Proof. To simplify our calculations, let $N := |\mathbb{P}(U)| = |\mathbb{P}(V)| = |\mathbb{P}(W)|$. Our approach will follow a similar structure as that of [14] and [2]. Remark that the number of *possible* walks of length ℓ in $\mathcal{G}(\mathcal{C})$ is precisely the number of walks of length ℓ in the complete tripartite graph $K_{N\times N\times N}$ with N being the size of each projective space. By proposition 2.3, this corresponds to a total of $(2N)^\ell + 2(-N)^\ell$ closed walks.

Furthermore, recall that by proposition 4.2 the probability that ℓ nodes form a closed walk is assumed to be $q^{-\ell n+l-1}$. Therefore, by our remark on the previous paragraph and by linearity of expectation,

$$\begin{aligned}\mathbb{E}_{\mathcal{C}\sim(U\otimes V\otimes W)^*}[M_\ell(\mathcal{C})] &\leq \sum_{(\hat{u}_1,\ldots,\hat{u}_\ell)\in S_\ell} \Pr[(\hat{u}_1,\ldots,\hat{u}_\ell) \text{ is a closed walk in } \mathcal{C}] \\ &= \frac{(2N)^\ell + 2(-N)^\ell}{q^{\ell n-\ell+1}}\end{aligned}$$

Where S_ℓ is the set of possible closed walks of length ℓ. Substituting N with $(q^n-1)/(q-1)$,

$$\begin{aligned}\mathbb{E}_{\mathcal{C}\sim(U\otimes V\otimes W)^*}[M_\ell(\mathcal{C})] &= \frac{(q^n-1)^\ell(2^\ell + 2(-1)^\ell)}{(q-1)^\ell q^{\ell n-\ell+1}} \\ &= \underset{q\to\infty}{\Theta}(1/q)\end{aligned}$$

□

In simple terms, the number of all possible arrangements of nodes and edges that yield closed walks of length ℓ in $\mathcal{C}$, is precisely the maximum number of closed walks that one could have in a graph. In particular, this corresponds to the number of closed walks of length ℓ in a complete tripartite graph. Then, by linearity of expectation, we multiply this count by the probability that each closed walk exists which we assume is the same for all closed walks.

Remark that when $\ell = 3$, the probability that three nodes form a triangle is precisely q^{-3n+2}, which coincides with the intermediate computation stated in the proof of proposition 2.3. Furthermore, the expected number of triangles is of the order of $1/q$, aligning with the findings in [14].

Similarly, this results represents a significant advantage over working with cycles of arbitrary length. Intuitively, as the length of a cycle increases, it becomes less likely to occur since each edge may only appear once. Moreover, as it can be inferred by corollary 3.1, the expected degree of an arbitrary node is less than one, which further reduces the likelihood of longer cycles. In contrast, closed walks do not impose the same uniqueness constraint on edges; they simply require that the walk returns to its starting point. Consequently, some degree of length-independence is preserved. Indeed, demonstrated in the previous proposition, the expected number of such walks remains on the order of $1/q$, up to a multiplicative factor of 2^ℓ.

Note, however, that there is a caveat when working with walks of arbitrary length. Indeed, as ℓ increases, there is a significant increase in the number of

equations needed to represent all closed walks of said length. For instance, as seen in section 4.2, in the case of $\ell = 4$, there are exactly six systems of equations needed to describe all possible closed walks of length four, whereas only three systems of equations are needed to describe all projective triangles. Naturally, this complicates the representation of all closed walks of a particular length in a compact algebraic form.

Proposition 4.4. (On the number of systems of equations) Heuristically, one needs $\Theta(2^\ell/\ell)$ systems of quadratic equations to algebraically characterize all closed walk of length ℓ.

Proof. Consider the complete graph of order 3, K_3, with labels $\mathbb{P}(U), \mathbb{P}(V)$, and $\mathbb{P}(W)$. A closed walk in K_3 that passes through $\mathbb{P}(U)$ a times, $\mathbb{P}(V)$ b times and $\mathbb{P}(W)$ c times defines a type of walk in $\mathcal{G}(\mathcal{C})$ that uses a nodes in $\mathbb{P}(U)$, b nodes in $\mathbb{P}(V)$ and c nodes in $\mathbb{P}(W)$. For instance, a closed walk in K_3 which passes through $\mathbb{P}(U)$ twice and through $\mathbb{P}(U)$ and $\mathbb{P}(W)$ once defines a type (or equivalence class) of closed walk in $\mathcal{G}(\mathcal{C})$ which uses two nodes in $\mathbb{P}(U)$, one node in $\mathbb{P}(V)$ and one node in $\mathbb{P}(W)$. If we were then to find all possible walks of this type, one would then solve the system:

$$\begin{cases} \mathcal{C}(u, v, -) = 0 \\ \mathcal{C}(u', v, -) = 0 \\ \mathcal{C}(u', -, w) = 0 \\ \mathcal{C}(u, -, w) = 0 \end{cases}$$

with $u, u' \in \mathbb{P}(U), v \in \mathbb{P}(V)$, and $w \in \mathbb{P}(W)$.

Therefore, for each equivalence class we define a unique system of equations, each with $n \times \ell$ equations: ℓ representing the length of the walk and n the number of elements in the basis of each vector space. The set of closed walks of a particular length therefore corresponds to the union of the solution sets obtained by working out each system of equations defined for each equivalence class.

By proposition 2.1 there are $2^\ell + 2(-1)^\ell$ closed walks of length ℓ in K_3. This number, however, does not quite represent the count of unique closed walks of a particular length, which is what interests us. Take, for example, the closed walk (u, v, u', w, u) displayed in the previous system of equations. The direction of the walk and whether the walk starts at u is irrelevant to us. Indeed, as explained in the previous paragraphs, it is only our interest that there is precisely one node in $\mathbb{P}(V)$, one node in $\mathbb{P}(W)$, and two in $\mathbb{P}(U)$ which are connected to v and w. To obtain the number of unique closed walks we therefore divide $2^\ell + 2(-1)^\ell$ by 2ℓ to account for all rotations and reflections of a given closed walk. We finally obtain that the number of systems of equations needed to algebraically characterize a closed walk of length ℓ is in $\Theta(2^\ell/\ell)$ [2]. □

[2] A more precise asymptotically equivalent formula may be derived using Burnside's counting lemma [13]

This suggests that the optimal balance lies with cycles of length 3 (triangles), since longer cycles offer only marginal improvements, and may even hinder the overall computation involved in solving non-linear systems of equations and determining the underlying isomorphism.

5. A Triangle-Based Isometry Finding Algorithm

In this section we shift the focus from counting special sub-structures to using them. We explain how a single projective triangle can serve as an anchor for recovering an isometry (A, B, C) between two tensors. In particular, we will restate the Ran-Samardjiska [14] algorithm, which (i) solves a compact system of quadratic equations to list all triangles of each tensor, (ii) picks a matching pair and, via a change of basis, pins the first rows of (A, B, C), and (iii) substitutes this information into the full 3-TIP equations to obtain the much lighter linear systems (4)-(6) that can be solved in near-linear time. No details are given regarding the lifting of collisions into full isometries from longer closed walks, since, as previously argued, a 3-closed walk (triangle) strikes the best balance between algebraic characterization and combinatorial abundance ($1/q$ frequency), making it the natural sweet spot for walk-based invariants. Now for the algorithm,

---

Input: two isomorphic tensors $\mathcal{C} \in (U \otimes V \otimes W)^*$ and $\mathcal{D} \in (U \otimes V \otimes W)^*$

Output: a triplet $(A, B, C) \in \mathrm{GL}(U) \times \mathrm{GL}(V) \times \mathrm{GL}(W)$ in the equivalence class of the isometry $(\mathcal{A}, \mathcal{B}, \mathcal{C})$.

1. Let $R = \mathbb{F}_q[x_2, \ldots, x_n, y_2, \ldots, y_m, z_2, \ldots, y_k]$, denote $x = (1, x_2, \ldots, x_n)$, $y = (1, y_2, \ldots, y_m)$, and $z = (1, z_2, \ldots, z_k)$. Solve the system of quadratic equations in R:

$$\begin{cases} \mathcal{C}(x, y, e_i^W) = 0 & \text{for } 1 \le i \le k \\ \mathcal{C}(x, e_i^V, z) = 0 & \text{for } 1 \le i \le m \\ \mathcal{C}(e_i^U, y, z) = 0 & \text{for } 1 \le i \le n \end{cases}$$

   Each element in the (eventually empty) set of solutions $S_{\mathcal{C}} = \{(x^*, y^*, z^*)\}$ is representative of a triangle for the tensor $\mathcal{C}$. Similarly, this process can be used to find the set of triangles[3] $S_{\mathcal{D}}$ of the tensor $\mathcal{D}$
2. If $|S_{\mathcal{C}}| = \emptyset$ or $|S_{\mathcal{D}}| = \emptyset$ abort. Otherwise, assume that $\mathcal{C}$ and $\mathcal{D}$ have unique triangles $T_{\mathcal{C}}$ and $T_{\mathcal{D}}$ (this process can be iterated for every pair of triangles in $S_{\mathcal{C}} \times S_{\mathcal{D}}$.) Since the triangles can be transformed to lie in any position via a suitable change of basis $(P_1, P_2, P_3) \in \mathrm{GL}(U) \times \mathrm{GL}(V) \times \mathrm{GL}(W)$, asume without loss of generality that $T_{\mathcal{C}} = (\hat{e}_1^U, \hat{e}_1^V, \hat{e}_1^W) = T_{\mathcal{D}}$. The final isomorphism will be conjugated with (P_1, P_2, P_3).

[3] with non-zero first coordinate

Since the isomorphism (A, B, C) maps $T_{\mathcal{C}}$ to $T_{\mathcal{D}}$ up to a scaling factor, A, B and C have the following form:

$$A = \begin{pmatrix} \lambda & A_{1,2} \\ 0_{(n-1)\times 1} & A_{2,2} \end{pmatrix} \quad B = \begin{pmatrix} \mu & B_{1,2} \\ 0_{(m-1)\times 1} & B_{2,2} \end{pmatrix} \quad C = \begin{pmatrix} \nu & C_{1,2} \\ 0_{(k-1)\times 1} & C_{2,2} \end{pmatrix}$$

By trilinearity we can rescale B and C so that $\mu = \nu = 1$ by scaling A by a factor $\mu \cdot \nu$. Without loss of generality, let us assume that such is the case. Notice that the vectors of the triangle are eigenvectors of (A, B, C), i.e. $Ae_1^U = \lambda e_1^U$, $Be_1^V = e_1^V$, $Ce_1^W = e_1^W$.

3. To find the coefficients of A, B, C, consider the following systems of equations:

$$\begin{cases} A \cdot A^{-1} = I_n = A^{-1} \cdot A \\ B \cdot B^{-1} = I_m = B^{-1} \cdot B \\ C \cdot C^{-1} = I_k = C^{-1} \cdot C \end{cases} \tag{1}$$

$$\begin{cases} \mathcal{C}(Ax, By, z) = \mathcal{D}(x, y, C^{-1}z) \\ \mathcal{C}(Ax, y, Cz) = \mathcal{D}(x, B^{-1}y, z) \\ \mathcal{C}(x, By, Cz) = \mathcal{D}(A^{-1}x, y, z) \end{cases} \quad \forall x \in \mathbb{F}_q^n, y \in \mathbb{F}_q^m, z \in \mathbb{F}_q^k \tag{2}$$

$$\begin{cases} \mathcal{C}(Ax, y, z) = \mathcal{D}(x, B^{-1}y, C^{-1}z) \\ \mathcal{C}(x, By, z) = \mathcal{D}(A^{-1}x, y, C^{-1}z) \\ \mathcal{C}(x, y, Cz) = \mathcal{D}(A^{-1}x, B^{-1}y, z) \end{cases} \quad \forall x \in \mathbb{F}_q^n, y \in \mathbb{F}_q^m, z \in \mathbb{F}_q^k \tag{3}$$

(2) and (3) provide a general (but expensive) solution for the 3-TIP. Nonetheless, thanks to the relationship described in step 4. we can find a much simpler almost linear system of equations:

$$\begin{cases} \mathcal{C}(Ax, e_1^V, z) = \mathcal{D}(x, e_1^V, C^{-1}z) \\ \mathcal{C}(x, e_1^V, Cz) = \mathcal{D}(A^{-1}x, e_1^V, z) \end{cases} \quad \forall x \in \mathbb{F}_q^n, z \in \mathbb{F}_q^k \tag{4}$$

$$\begin{cases} \mathcal{C}(Ax, y, e_1^W) = \mathcal{D}(x, B^{-1}y, e_1^W) \\ \mathcal{C}(x, By, e_1^W) = \mathcal{D}(A^{-1}x, y, e_1^W) \end{cases} \quad \forall x \in \mathbb{F}_q^n, y \in \mathbb{F}_q^m \tag{5}$$

$$\begin{cases} \mathcal{C}(\lambda e_1^U, By, z) = \mathcal{D}(e_1^U, y, C^{-1}z) \\ \mathcal{C}(\lambda e_1^U, y, Cz) = \mathcal{D}(e_1^U, B^{-1}y, z) \end{cases} \quad \forall y \in \mathbb{F}_q^m, z \in \mathbb{F}_q^k \tag{6}$$

Solving for A, B, C we find a matrix triplet in the equivalence class of the sought isomorphism.

6. Tensor Graph Implementation

In this section we translate the theory developed so far into a hands-on software platform that builds, explores and visualizes the tripartite graph of any 3-tensor. In particular, this tool was written to

- Give researchers a sandbox where graphical invariant hypotheses can be tested on concrete instances.
- Supply pictures and statistics that underpin empirical claims.
- Offer a rudimentary reference implementation of core routines: tensor evaluation, edge testing, cycle enumeration and isometry application.

Powered by a flexible `SageMath` / `NetworkX` framework, a single command can generate a random tensor, filter vertices by degree, colour every k-cycle, apply arbitrary (A, B, C) isometries and export the resulting plot or PNG snapshot.

Yet, the program is designed for the small scale. Each run still enumerates every candidate edge, yielding an $O(q^{2n}n^3)$ workload before cycle-finding begins, while the vertex set itself swells as $\Theta(q^{3n})$ when $n = m = k$. Memory and screen space therefore explode well below cryptographic parameter sizes, making it infeasible to draw or interactively explore graphs of real-world interest. In this section we consequently restrict experiments to toy instances (typically $n \leq 4$, $q \leq 17$) which are large enough to show genuine sparsity, cycle structure and isometry effects, but small enough to stay within laptop limits. The implementation is thus invaluable for intuition, debugging and expounding, while its main limitation is precisely what keeps the 3-Tensor Isomorphism Problem hard: the combinatorial blow-up that accompanies higher dimensions and field sizes.

6.1. Overview

The implemented tensor graph software includes essential functionalities for analyzing and displaying the tri-partite graph of a 3-tensor. Specifically, the system supports:

- Construction and visualization of graphs from randomly generated 3-tensors.
- Evaluation of tensors at given points.
- Testing edge conditions among vertices to establish connectivity in the tensor graph.
- Identification and visualization of graph cycles of specified lengths.
- Application of random isometries to tensors and subsequent visualization of the resulting transformed graphs.
- Filtering of graph vertices based on node degree constraints to simplify and clarify visualization.

These functionalities collectively facilitate comprehensive analyses of tensor graph properties relevant to the cryptanalisis of the 3-TIP.

6.2. Functions

- `tensor_value(T,u,v,w)`: Evaluates tensor $\mathcal{T}$ at points $(u, v, w) \in (U \times V \times W)$ by performing a triple summation over their coordinates.
- `is_edge_UV(T,u,v)`: tests if $\mathcal{T}(u, v, -) = 0$. Similar for `is_edge_UW(T,u,w)` and `is_edge_VW(T,v,w)` Check whether the partial evaluations vanish, indicating the presence of an edge between nodes in respective subspaces.
- `tensor_to_graph(T,n,m,k,F)`: Builds a graph for tensor $\mathcal{T}$ of dimension $n \times m \times k$ over the finite field $\mathcal{F}$. It begins by listing the elements of the projective spaces for U, V, W. Then, it succinctly relabels the elements in $\mathcal{V}_{\mathcal{T}}$ and finally tests the existence of an edge for each pair of point.
- `apply_isometry(T,A,B,C)`: Given a tensor $\mathcal{T}$ and $(A, B, C) \in \mathrm{GL}(U) \times \mathrm{GL}(V) \times \mathrm{GL}(W)$, outputs $\mathcal{T}' = \mathcal{T} \circ (A, B, C)$.

6.3. Edge Testing

During `is_edge_UV` (respectively UW and VW), to determine if $(u, v) \in \mathcal{E}_{\mathcal{C}}$ we test:

$$\forall w \in W, \mathcal{C}(u, v, w) = \sum_{i=0}^{n} \sum_{j=0}^{m} \sum_{l=0}^{k} \mathcal{C}_{i,j,l} u_i v_j w_k = 0$$

This is equivalent to the following implemented test:

$$\forall l \in [1, k], \sum_{i=1}^{n} \sum_{j=1}^{m} \mathcal{C}_{i,j,l} \cdot u_i \cdot v_j = 0$$

Indeed, given $(u, v) \in \mathbb{P}(U) \times \mathbb{P}(V)$, denote

$$C_u^w := (\sum_{i=1}^{n} \sum_{j=1}^{m} \mathcal{C}_{i,j,l} \cdot u_i \cdot v_j)_{l=1}^{k} \in \mathbb{F}_q^k$$

For all $w \in W$,

$$\begin{aligned} \mathcal{C}(u, v, w) &= \sum_{i=1}^{k} w_i \cdot (C_v^w)_i \\ &= \langle w, C_v^w \rangle \end{aligned}$$

Thus, $\forall w \in W, \mathcal{C}(u, v, w) = 0 \iff C_v^w = (0 \dots 0) \in \mathbb{F}_q^k$ by positive definiteness of the scalar product.

6.4. Cycle Finding

The cycle finding functionality employs a depth-first search (DFS) algorithm (`find_cycles_of_length_c`) that systematically explores all walks of length c starting from each vertex. It identifies all unique simple cycles of a specified length by testing if the final vertex matches the starting one, and no other vertex has been visited more than once. Furthermore, it ensures that cycles are unique by adding only those starting from the smallest vertex label. This avoids redundant cycles due to cyclic permutations or reversals.

Graphically, this implementation supports highlighting all found cycles by displaying all nodes in a specific cycle in red. Furthermore, the `--loose` flag allows to also identify and highlight cycles of length $3 \leq c' \leq c$.

6.5. Graph Visualization

Graph visualization is handled using NetworkX and matplotlib libraries through the function `graph_display`. This tool converts SageMath graphs into NetworkX-compatible graphs and visualizes them with:

- Optional vertex labeling.
- Highlighted vertices involved in cycles of specific length.
- Support for loose cycle highlighting (cycles of length within a specified range).
- Capability to save graph visualizations as PNG images.

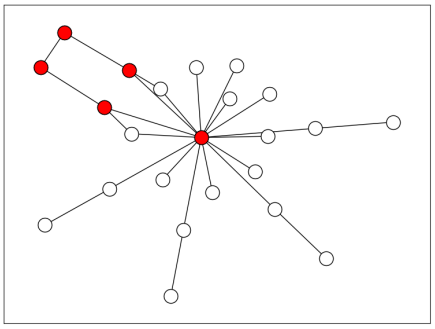

Figure 8: Sample execution output for $k = 3, q = 7, c = 5$. Vertices of degree $d \leq 2$ filtered out

6.6. Total Complexity

Recall that in section 3 we showed that the size of each projective space is of the order q^n. Therefore, considering both the edge checks and the iteration over the Cartesian product $\mathbb{P}(\mathbb{F}_q^n) \times \mathbb{P}(\mathbb{F}_q^n)$, the computational complexity of computing the graph becomes $O(q^{2n} \cdot n^3)$.

6.7. Sample Execution

As described in the README.md page of the repository. The implementation takes command line arguments to initialize the graph parameters and execution conditions. Table A.3 in the appendix summarizes all command line arguments and their respective syntax. Figure 8 shows a sample execution given the following instruction:

```
python3 main.py -k=3 -q=7 -c=5 --deg_lbound 2
```

For further details and specifications see: `https://github.com/david-pulido/3TI_Graph_Visualizer`.

7. Discussion and Future Directions

The findings in this report open several avenues for further exploration of the 3-Tensor Isomorphism Problem (3-TIP). One may notice, for instance, that while the core theory imposes no formal constraint on the three dimensions, most results, including ours, specialise to the balanced case $n = m = k$ to keep statements and proofs readable. We begin this section on future work by remarking on new graphical substructures that emerge in exactly the opposite unexplored setting where $n, m \neq k$. They were discovered empirically with the visualizer toolkit of section 6 and appear promising for refining both attack and hardness estimates.

We then propose a shift in perspective: instead of encoding a tensor as the tripartite graph used throughout the report, we recast it as a 3-uniform hypergraph whose hyperedges are precisely the non-zero entries of the tensor. This model preserves the full three-way symmetry of the data and unlocks a richer range of combinatorial invariants that have no analogue in ordinary graphs: star sizes, transversal numbers, Helly properties, hypergraph Laplacian spectra. The discussion below sketches how these hypergraph-level features could be harnessed for isomorphism testing and, ultimately, for recovering the hidden linear maps. Readers who are new to hypergraph terminology will find a short primer in appendix Appendix A.3.

7.1. Unequal Dimension Case

It is important to highlight that the general structure of a tensor graph varies significantly depending on whether the dimensions n, m, k are equal or if at least one dimension differs. Empirical observations using the tensor graph visualizer presented in section 6 indicate that when the dimensions are equal,

after filtering out nodes of degree zero, the resulting graph typically consists of one large connected component that encompasses most nodes, accompanied by a few smaller outlier components. A high-resolution example illustrating this structure is provided in appendix Appendix A.1, figure A.9.

In contrast, when the dimensions differ, the main connected component exhibits a distinct, tree-like structure. Additionally, in such cases, it has been observed that random tensor graphs generally contain numerous smaller connected components, each typically consisting of two or three nodes without cycles. An example depicting this phenomenon is presented in appendix Appendix A.1, figure A.10. It should be noted, however, that these findings are preliminary and specifically pertain to scenarios involving relatively small values of n, m, k, q.

7.2. Tensor Hypergraphs

In the study of the 3-TIP, a 3-tensor is intrinsically a tri-linear object, so it could be argued that the most faithful combinatorial model is not the tripartite graph $\mathcal{G}(\mathcal{C})$ used in the body of this report (which forgets one index at a time) but a 3-*uniform,* 3-*partite hypergraph* that records all three indices simultaneously and, potentially, unlocks a series of new compelling invariants.

Definition 7.1. Let $\mathcal{C} \in (U \otimes V \otimes W)^*$. We define the 3-uniform 3-partite hypergraph associated to a 3-tensor, noted $\mathcal{H}(\mathcal{C})$, as

$$\mathcal{H}(\mathcal{C}) := (\mathcal{V}_{\mathcal{C}}^H, \mathcal{E}_{\mathcal{C}}^H)$$

Where $\mathcal{V}_{\mathcal{C}}^H := U \cup V \cup W$ and

$$\mathcal{E}_{\mathcal{C}}^H := \{(u, v, w) \in U \times V \times W \mid \mathcal{C}(u, v, w) = 0\}$$

One quickly observes that for every edge $(\hat{u}, \hat{v})$ in the standard tensor graph $\mathcal{G}(\mathcal{C})$, there is an associated hyperedge $(u, v, w) \in \mathcal{E}_{\mathcal{C}}^H$ for all $w \in W$. However, there can exist hyperedges $(u, v, w) \in \mathcal{E}_{\mathcal{C}}^H$ such that none of the edges $(\hat{u}, \hat{v}), (\hat{u}, \hat{w})$, or $(\hat{v}, \hat{w})$ belong to $\mathcal{E}_{\mathcal{C}}$. Thus, this hypergraph construction strictly refines the information contained in $\mathcal{G}(\mathcal{C})$. In fact, $\mathcal{H}(\mathcal{C})$ perfectly encodes the support of $\mathcal{C}$.

Along these lines, beyond the adjacency, incidence, and Laplacian matrices defined in appendix Appendix A.3, we introduce another important object within the context of a 3-uniform, 3-partite hypergraph: the *support tensor*. This tensor is defined as

$$\mathcal{S}(\mathcal{C}) : \mathbb{F}_q^n \times \mathbb{F}_q^m \times \mathbb{F}_q^k \to \mathbb{F}_2$$

where $\mathcal{S}(u, v, w) := 1$ if and only if $(u, v, w) \in \mathcal{E}_{\mathcal{C}}^H$. This approach may initially appear as lateral movement. However, since each hyperedge (u, v, w) occurs with probability $1/q$, the resulting tensor is highly sparse, which might facilitate certain algebraic analyses.

Beyond this construction, which the reader may or may not find more natural, a further intriguing question arises regarding the identification of new or

different invariants. For instance, in appendix Appendix A.3, we introduce the notions of star, Laplacian, and hypergraph entropy, as well as briefly mention the Helly and strong Helly properties, along with matching and transversal numbers.

All these invariants could serve as distinguishing factors to determine whether two tensors are isomorphic. Some invariants might even enable the recovery of the full isometry. For example, the star of a subset of nodes shows promise for recovering the complete isometry. Furthermore, methods such as Bouillaguet's [5] and subsequently Narayanan–Qiao–Tang's [12] birthday paradox approach could still be applied within the hypergraph framework, preserving the same square-root complexity while leveraging strictly richer local data intrinsic to the hypergraph representation.

8. Conclusion

This report set out to study the 3-Tensor Isomorphism Problem from three angles: pure combinatorics, concrete algorithms, and practical tooling. Section 3 quantified just how sparse a tensor graph is, showing that while the vertex count explodes as $\Theta(q^{3n})$, the expected edge count grows only as $\Theta(q^{3n/2})$. Section 3 then explored the geometry hidden inside that sparsity. We extended the 3-cycle result to 4-closed walks. Then, we proved an ℓ-independent bound $\mathbb{E}[M_\ell(C)] = \Theta(1/q)$ for every closed walk of length $\ell \geq 3$, and established that $\Theta(2^\ell/\ell)$ systems of quadratic equations describe all such walks, identifying triangles as the natural sweet spot for isometry attacks. Building on that insight, section 5 restated and clarified the triangle-driven solver of Ran and Samardjiska, arguing that it offers a good complexity-versus-accuracy balance. Finally, section 6 delivered an open-source SageMath/NetworkX visualizer that spins up random tensors, colours their cycles, and applies arbitrary isometries, which is indispensable for intuition and didactic purposes, even if it cannot scale to cryptographic parameters.

Section 7 then opened two promising directions. First, preliminary experiments hint at a qualitative geometric change when at least one of the three dimensions differs ($n = m \neq k$), fragmenting the core of the graph into tree-like micro-components that deserve separate analysis. Second, modeling a tensor as a 3-uniform, 3-partite hypergraph retains the full trilinear symmetry and unlocks richer invariants: star ranks, Laplacian entropy, Helly numbers, matching-transversal gaps, and more. Certain invariants, such as star-cardinality histograms, could already be computed at real-world parameters. Other global invariants remain a target for further analysis.

Taken together, these results supply both a sharper picture of why 3-TIP instances look hard and a toolkit for visualizing and exploring that hardness algorithmically. The hope is that the counting techniques and results, as well as the publicly released code, will inform future research, either to strengthen the MEDS and ALTEQ proposals or to inspire the next generation of attacks and defenses built on tensor (hyper)graph structure.

References

[1] D. J. Bernstein, A. Hülsing, S. Kölbl, R. Niederhagen, J. Rijneveld, and P. Schwabe. The sphincs+ signature framework. In *Proceedings of the 2019 ACM SIGSAC conference on computer and communications security*, pages 2129–2146, 2019.

[2] W. Beullens. Graph-theoretic algorithms for the alternating trilinear form equivalence problem. In *Annual International Cryptology Conference*, pages 101–126. Springer, 2023.

[3] M. Bläser, D. H. Duong, A. K. Narayanan, T. Plantard, Y. Qiao, A. Sipasseuth, and G. Tang. The alteq signature scheme: Algorithm specifications and supporting documentation. *NIST PQC Submission*, 2023.

[4] J. Bos, L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, J. M. Schanck, P. Schwabe, G. Seiler, and D. Stehlé. Crystals-kyber: a cca-secure module-lattice-based kem. In *2018 IEEE European Symposium on Security and Privacy (EuroS&P)*, pages 353–367. IEEE, 2018.

[5] C. Bouillaguet, P.-A. Fouque, and A. Véber. Graph-theoretic algorithms for the "isomorphism of polynomials" problem. In *Advances in Cryptology–EUROCRYPT 2013: 32nd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Athens, Greece, May 26-30, 2013. Proceedings 32*, pages 211–227. Springer, 2013.

[6] A. Bretto. Hypergraph theory. *An introduction. Mathematical Engineering. Cham: Springer*, 1:209–216, 2013.

[7] T. Chou, R. Niederhagen, E. Persichetti, T. H. Randrianarisoa, K. Reijnders, S. Samardjiska, and M. Trimoska. Take your meds: digital signatures from matrix code equivalence. In *International conference on cryptology in Africa*, pages 28–52. Springer, 2023.

[8] A. Couvreur and C. Levrat. Highway to hull: An algorithm for solving the general matrix code equivalence problem. Cryptology ePrint Archive, Paper 2025/596, 2025.

[9] J. Grochow and Y. Qiao. On the complexity of isomorphism problems for tensors, groups, and polynomials i: tensor isomorphism-completeness. *SIAM Journal on Computing*, 52(2):568–617, 2023.

[10] R. Horn and C. Johnson. *Topics in Matrix Analysis*. Cambridge University Press, Cambridge, 1991.

[11] V. Lyubashevsky, L. Ducas, E. Kiltz, T. Lepoint, P. Schwabe, G. Seiler, D. Stehlé, and S. Bai. Crystals-dilithium. *Algorithm Specifications and Supporting Documentation*, 2020.

[12] A. K. Narayanan, Y. Qiao, and G. Tang. Algorithms for matrix code and alternating trilinear form equivalences via new isomorphism invariants. Cryptology ePrint Archive, Paper 2024/368, 2024.

[13] M. T. Nasseef. Counting symmetries with burnside's lemma and polya's theorem. *European journal of pure and applied mathematics*, 9(1):84–113, 2016.

[14] L. Ran and S. Samardjiska. Rare structures in tensor graphs: Bermuda triangles for cryptosystems based on the tensor isomorphism problem. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 66–96. Springer, 2024.

[15] O. Regev. An efficient quantum factoring algorithm. *Journal of the ACM*, 72(1):1–13, 2025.

[16] P. W. Shor. Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM review*, 41(2):303–332, 1999.

[17] D. B. West et al. *Introduction to graph theory*, volume 2. Prentice hall Upper Saddle River, 2001.

Appendix A. Appendix

Appendix A.1. Tensor Graph Visualization Examples

The tensor graph shown in figure A.9 can be obtained by entering the following tensor into the graph visualizer presented in Section 6.

$$\left[\begin{pmatrix} 4 & 4 & 4 & 1 & 2 \\ 3 & 1 & 2 & 4 & 0 \\ 0 & 4 & 4 & 2 & 0 \\ 3 & 1 & 0 & 1 & 0 \\ 0 & 1 & 4 & 1 & 0 \end{pmatrix}, \begin{pmatrix} 2 & 0 & 4 & 2 & 0 \\ 1 & 4 & 0 & 1 & 0 \\ 4 & 3 & 4 & 0 & 2 \\ 3 & 2 & 1 & 2 & 3 \\ 2 & 0 & 2 & 3 & 0 \end{pmatrix}, \begin{pmatrix} 0 & 0 & 0 & 1 & 3 \\ 4 & 2 & 3 & 0 & 1 \\ 4 & 3 & 4 & 2 & 4 \\ 3 & 4 & 0 & 1 & 4 \\ 0 & 3 & 2 & 3 & 4 \end{pmatrix}, \right.$$
$$\left. \begin{pmatrix} 3 & 3 & 1 & 1 & 4 \\ 1 & 1 & 3 & 0 & 1 \\ 3 & 0 & 3 & 3 & 2 \\ 4 & 1 & 0 & 4 & 1 \\ 2 & 0 & 2 & 3 & 0 \end{pmatrix}, \begin{pmatrix} 3 & 2 & 0 & 4 & 1 \\ 4 & 2 & 1 & 1 & 3 \\ 4 & 2 & 1 & 0 & 3 \\ 3 & 1 & 2 & 4 & 1 \\ 1 & 3 & 4 & 0 & 2 \end{pmatrix} \right]$$

Similarly, the tensor shown in figure A.10 can be obtained with the tensor:

$$\left[\begin{pmatrix} 3 & 2 & 0 & 0 \\ 4 & 2 & 3 & 3 \\ 3 & 0 & 3 & 2 \\ 2 & 1 & 1 & 3 \end{pmatrix}, \begin{pmatrix} 1 & 0 & 2 & 2 \\ 1 & 3 & 1 & 0 \\ 4 & 2 & 4 & 0 \\ 4 & 2 & 3 & 4 \end{pmatrix}, \begin{pmatrix} 0 & 0 & 2 & 4 \\ 1 & 2 & 3 & 2 \\ 0 & 3 & 3 & 1 \\ 4 & 3 & 2 & 0 \end{pmatrix}, \right.$$
$$\left. \begin{pmatrix} 4 & 1 & 3 & 3 \\ 3 & 4 & 1 & 0 \\ 1 & 3 & 0 & 4 \\ 0 & 0 & 4 & 4 \end{pmatrix}, \begin{pmatrix} 4 & 0 & 3 & 1 \\ 4 & 4 & 3 & 2 \\ 4 & 0 & 0 & 1 \\ 0 & 3 & 3 & 2 \end{pmatrix} \right]$$

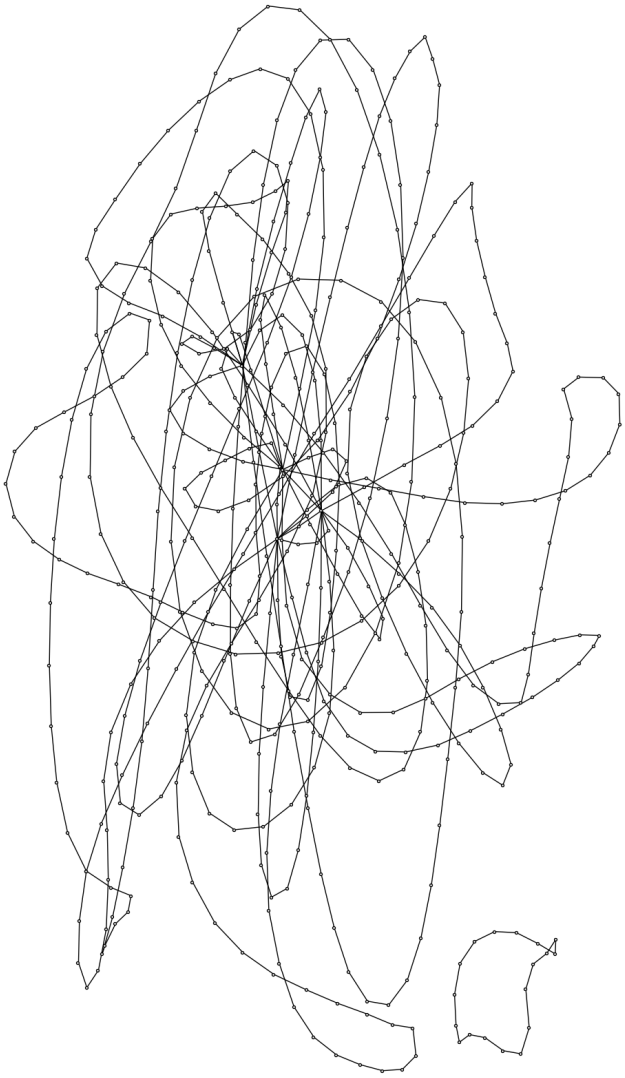

Figure A.9: Example of tensor graph with $n = m = k = 5$ and $q = 5$

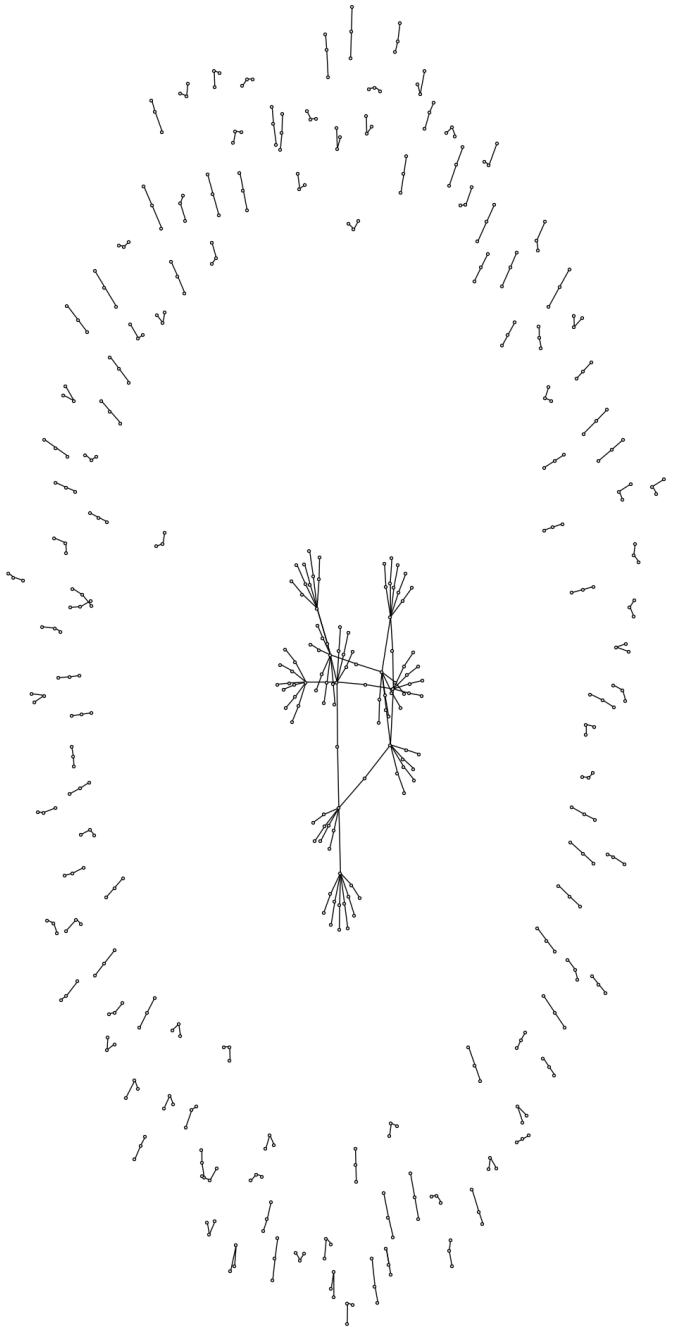

Figure A.10: Example of tensor graph with $n = 5, m = k = 4$, and $q = 5$

Option	Description	Default
-n N	Dimension n for the first vector space	4
-m M	Dimension m for the second vector space	4
-k K	Dimension k for the third vector space	4
-q Q	Prime field size	5
-c C	Highlights all cycles of length C in the final graph	0
–loose	Highlights all cycles of length $2 < c' \leq C$	false
–deg_lbound D	Filters out all nodes of degree less or equal than specified	0
–deg_ubound D	Filters out all nodes of degree greater or equal than specified	1000
–isolated_nodes	Displays nodes of degree zero on the final graph	false
–labeled	Show graph with vertex labels	false
–verbose	Show extra info on terminal	false
–isometry	Applies a random isometry to the original tensor and displays it	false
–no_visualization	Prevents the display of a new tab with the resulting tensor graph	false
–minimal	Only prints in terminal the random tensor, and, if enabled, all cycles of various lengths	false
–load_tensor	Loads tensor from file instead of generating a random one	””
–load_graph	Loads tensor graph from file instead of calculating one given a random tensor	””

Table A.3: Command-line options and their descriptions

Appendix A.3. Hypergraph Theory Primer

We will now give a short introduction to hypergraph theory. The definitions and results listed in this appendix are based on the work of Bretto, section 1.3 [6].

Definition Appendix A.1. A **hypergraph** H is an ordered pair (V, E) where V is a set of vertices and E is a family of subsets of V called *hyperedges*.

Definition Appendix A.2. Given $H = (V, E)$ and a node $v \in V$, the **star** of v, noted $S_H(v)$ is the set of all edges containing v i.e.

$$S_H(v) := \{e \in E \mid v \in e\}$$

The **degree** of a node $v \in V$ is then the size of its star, $d_H(v) := |S_H(v)|$. The concept of maximum degree or maximum star size naturally follows these definitions.

Definition Appendix A.3. Let $k \in \mathbb{N}^*$. A hypergraph H is **k-uniform** if all its hyperedges are of size k

Notice that in contrast to ordinary graphs, in a hypergraph, an edge can join any number of vertices. Thus, a graph can be seen as a special case of a hypergraph, in particular a 2-uniform hypergraph.

Definition Appendix A.4. Naturally following the definition for regular graphs, a hypergraph H is **k-partite** if its vertex set V can be partitioned into k disjoint parts $V_1, \ldots, V_k$ so that no edge contains more than one vertex from the same part. As discussed in section 7, a 3-tensor can be encoded as a 3-uniform 3-partite hypergraph.

Definition Appendix A.5. The coordinates of the **incidence matrix** $B \in \mathcal{M}_{|V|\times|E|}(\mathbb{F}_q)$ of a hypergraph $H = (V, E)$ is defined as follows:

$$B_{v,e} := \begin{cases} 1 \text{ iff } v \in e \\ 0 \text{ otherwise} \end{cases}$$

Similarly, its **adjacency matrix** $A \in \mathcal{M}_{|V|\times|V|}(\mathbb{F}_q)$ is defined as,

$$A_{i,j} := \begin{cases} 0 \text{ if } i = j \\ |\{e \in E \mid i, j \in e\}| \text{ otherwise} \end{cases}$$

Given these definitions we can introduce another special invariant referred to as the combinatorial Laplacian.

Definition Appendix A.6. Let $H = (V, E)$ of adjacency matrix A. Define $D := \text{diag}(\deg_H(v))$. The **Laplacian matrix** of H is defined as,

$$L(H) := D - A$$

From this definition we notice that $L(H)$ is symmetric and positive semidefinite of eigenvalues $0 \leq \lambda_1, \ldots, \lambda_{|V|=n}$. We can then define the entropy associated to a hypergraph as

$$I(H) = -\sum_{i=1}^{|V|} p_i \log_2(p_i)$$

where

$$p_i := \frac{\lambda_i}{\mathrm{Tr}(D)}$$

There are many other invariants that arise from the definition of a hypergraph, such as Helly and Helly strong properties, matching, chromatic, and transerval numbers. They will not be discussed in this appendix, but can be found in [6].